

Chapter 6

Numerical Representations

Consider the usual representations of lists and natural numbers, along with several typical functions on each data type.

datatype α List = Nil Cons of $\alpha \times \alpha$ List	datatype Nat = Zero Succ of Nat
fun tail (Cons (x , xs)) = xs	fun pred (Succ n) = n
fun append (Nil, ys) = ys append (Cons (x , xs), ys) = Cons (x , append (xs , ys))	fun plus (Zero, n) = n plus (Succ m , n) = Succ (plus (m , n))

Other than the fact that lists contain elements and natural numbers do not, these two implementations are virtually identical. This suggests a strong analogy between representations of the number n and representations of container objects of size n . Functions on the container strongly resemble arithmetic functions on the number. For example, inserting an element resembles incrementing a number, deleting an element resembles decrementing a number, and combining two containers resembles adding two numbers. This analogy can be exploited to design new implementations of container abstractions — simply choose a representation of natural numbers with certain desired properties and define the functions on the container objects accordingly. Call an implementation designed in this fashion a *numerical representation*.

The typical representation of lists can be viewed as a numerical representation based on unary numbers. However, numerical representations based on binary numbers are also common; the best known of these is the binomial queues of Vuillemin [Vui78]. Incrementing a unary number takes $O(1)$ time, so inserting an element into a unary representation also usually takes $O(1)$ time. However, adding two unary numbers takes $O(n)$ time, so combining two containers in a unary representation takes $O(n)$ time. Binary numbers improve the time

required for addition (and hence the time required to combine two containers) to $O(\log n)$, but also slow the time required to increment a number or insert an element to $O(\log n)$. In this chapter, we consider several variations of binary numbers that achieve the best of both worlds by supporting the increment function in $O(1)$ time and addition in $O(\log n)$ time. Numerical representations based on these variations naturally support inserting an element in $O(1)$ time and combining two containers in $O(\log n)$ time.

Example abstractions for which numerical representations are particularly useful include *random-access lists* (also known as *flexible arrays*) and *heaps* (also known as *priority queues*).

6.1 Positional Number Systems

A *positional number system* [Knu73] is a notation for writing a number as a sequence of digits $b_0 \dots b_{m-1}$. The digit b_0 is called the *least significant digit* and the digit b_{m-1} is called the *most significant digit*. Except when writing ordinary, decimal numbers, we will always write sequences of digits from least significant to most significant.

Each digit b_i has weight w_i , so the value of the sequence $b_0 \dots b_{m-1}$ is $\sum_{i=0}^{m-1} b_i w_i$. For any given positional number system, the sequence of weights is fixed, as is the set of digits D_i from which each b_i is chosen. For unary numbers, $w_i = 1$ and $D_i = \{1\}$ for all i , and for binary numbers $w_i = 2^i$ and $D_i = \{0, 1\}$. (By convention, we write all digits in typewriter font except for ordinary, decimal digits.) A number is said to be written in base B if $w_i = B^i$ and $D_i = \{0, \dots, B-1\}$. Usually, but not always, weights are increasing sequences of powers, and the set D_i is the same for every digit.

A number system is said to be *redundant* if there is more than one way to represent some numbers. For example, we can obtain a redundant system of binary numbers by taking $w_i = 2^i$ and $D_i = \{0, 1, 2\}$. Then the decimal number 13 can be written 1011, or 1201, or 122. If we allow trailing 0s, then almost all positional number systems are redundant, since $b_0 \dots b_{m-1}$ is always equivalent to $b_0 \dots b_{m-1}0$. Therefore, we disallow trailing 0s.

Computer representations of positional number systems can be *dense* or *sparse*. A dense representation is simply a list (or some other kind of sequence) of digits, including those digits that happen to be 0. A sparse representation, on the other hand, includes only non-zero digits. It must then include information on either the rank (i.e., the index) or the weight of each digit. For example, Figure 6.1 shows two different representations of binary numbers in Standard ML— one dense and one sparse — along with several representative functions on each.

```

structure Dense =
struct
  datatype Digit = Zero | One
  type Nat = Digit list      (* increasing order of significance, no trailing Zeros *)

  fun inc [] = [One]
    | inc (Zero :: ds) = One :: ds
    | inc (One :: ds) = Zero :: inc ds    (* carry *)

  fun dec [One] = []
    | dec (One :: ds) = Zero :: ds
    | dec (Zero :: ds) = One :: dec ds    (* borrow *)

  fun add (ds, []) = ds
    | add ([], ds) = ds
    | add (d :: ds1, Zero :: ds2) = d :: add (ds1, ds2)
    | add (Zero :: ds1, d :: ds2) = d :: add (ds1, ds2)
    | add (One :: ds1, One :: ds2) = Zero :: inc (add (ds1, ds2))    (* carry *)
end

```

```

structure SparseByWeight =
struct
  type Nat = int list      (* increasing list of weights, each a power of two *)
  (* add a new weight to a list, recurse if weight is already present *)
  fun carry (w, []) = [w]
    | carry (w, ws as w' :: rest) = if w < w' then w :: ws else carry (2*w, rest)
  (* borrow from a digit of weight w, recurse if weight is not present *)
  fun borrow (w, ws as w' :: rest) = if w = w' then rest else w :: borrow (2*w, ws)

  fun inc ws = carry (1, ws)
  fun dec ws = borrow (1, ws)

  fun add (ws, []) = ws
    | add ([], ws) = ws
    | add (m as w1 :: ws1, n as w2 :: ws2) =
      if w1 < w2 then w1 :: add (ws1, n)
      else if w2 < w1 then w2 :: add (m, ws2)
      else carry (2*w1, add (ws1, ws2))
end

```

Figure 6.1: Two implementations of binary numbers.

6.2 Binary Representations

Given a positional number system, we can implement a numerical representation based on that number system as a sequence of trees. The number and sizes of the trees representing a collection of size n are governed by the representation of n in the positional number system. For each weight w_i , there are b_i trees of that size. For example, the binary representation of 73 is 1001001, so a collection of size 73 in a binary numerical representation would comprise three trees, of sizes 1, 8, and 64, respectively.

Trees in numerical representations typically exhibit a very regular structure. For example, in binary numerical representations, all trees have sizes that are powers of 2. Three common kinds of trees that exhibit this structure are *complete binary leaf trees* [KD96], *binomial trees* [Vui78], and *pennants* [SS90].

Definition 6.1 (Complete binary leaf trees) A complete binary tree of rank 0 is a leaf and a complete binary tree of rank $r > 0$ is a node with two children, each of which is a complete binary tree of rank $r - 1$. A leaf tree is a tree that contains elements only at the leaves, unlike ordinary trees that contain elements at every node. A complete binary tree of rank r has $2^{r+1} - 1$ nodes, but only 2^r leaves. Hence, a complete binary leaf tree of rank r contains 2^r elements.

Definition 6.2 (Binomial trees) A binomial tree of rank r is a node with r children $c_1 \dots c_r$, where c_i is a binomial tree of rank $r - i$. Alternatively, a binomial tree of rank $r > 0$ is a binomial tree of rank $r - 1$ to which another binomial tree of rank $r - 1$ has been added as the leftmost child. From the second definition, it is easy to see that a binomial tree of rank r contains 2^r nodes.

Definition 6.3 (Pennants) A pennant of rank 0 is a single node and a pennant of rank $r > 0$ is a node with a single child that is a complete binary tree of rank $r - 1$. The complete binary tree contains $2^r - 1$ elements, so the pennant contains 2^r elements.

Figure 6.2 illustrates the three kinds of trees. Which kind of tree is superior for a given data structure depends on the properties the data structure must maintain, such as the order in which elements should be stored in the trees. A key factor in the suitability of a particular kind of tree for a given data structure is how easily the tree supports functions analogous to carries and borrows in binary arithmetic. When simulating a carry, we *link* two trees of rank r to form a tree of rank $r + 1$. Symmetrically, when simulating a borrow, we *unlink* a tree of rank $r > 0$ to obtain two trees of rank $r - 1$. Figure 6.3 illustrates the link operation (denoted $\oplus$) on each of the three kinds of trees. Assuming that elements are not rearranged, each of the three kinds of trees can be linked or unlinked in $O(1)$ time.

We next describe two existing data structures in terms of this framework: the one-sided flexible arrays of Kaldewaij and Dielissen [KD96], and the binomial queues of Vuillemin [Vui78, Bro78].

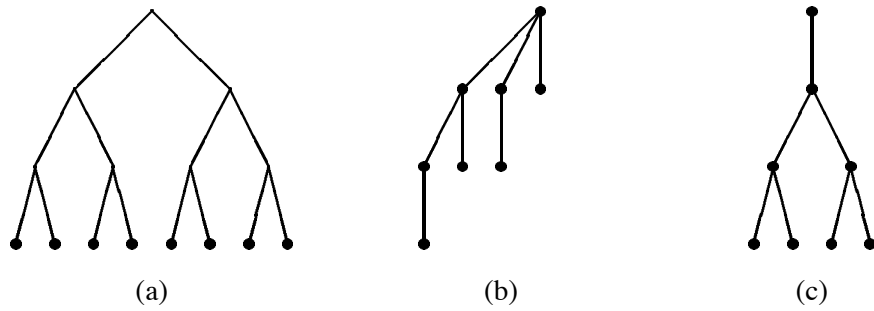


Figure 6.2: Three trees of rank 3: (a) a complete binary leaf tree, (b) a binomial tree, and (c) a pennant.

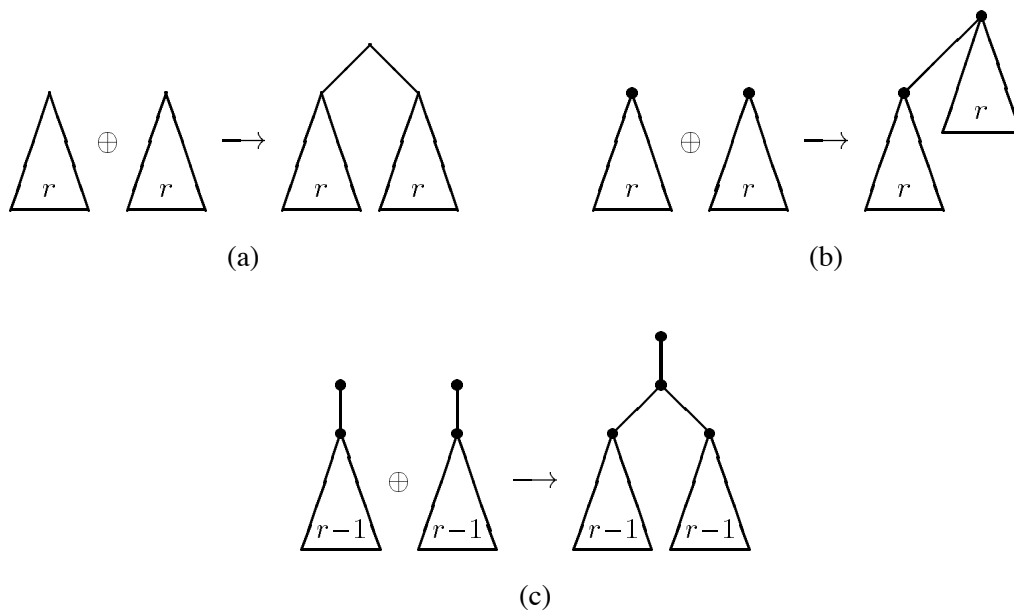


Figure 6.3: Linking two trees of rank r to obtain a tree of rank $r + 1$ for (a) complete binary leaf trees, (b) binomial trees, and (c) pennants.

```

signature RANDOMACCESSLIST =
sig
  type  $\alpha$  RList
  exception EMPTY and INDEX
  val empty   :  $\alpha$  RList
  val isEmpty :  $\alpha$  RList  $\rightarrow$  bool
  val cons    :  $\alpha \times \alpha$  RList  $\rightarrow \alpha$  RList
  val head    :  $\alpha$  RList  $\rightarrow \alpha$  (* raises EMPTY if list is empty *)
  val tail    :  $\alpha$  RList  $\rightarrow \alpha$  RList (* raises EMPTY if list is empty *)
  val lookup  :  $\alpha$  RList  $\times$  int  $\rightarrow \alpha$  (* raises INDEX if out of bounds *)
  val update  :  $\alpha$  RList  $\times$  int  $\times \alpha \rightarrow \alpha$  RList (* raises INDEX if out of bounds *)
end

```

Figure 6.4: Signature for random-access lists.

6.2.1 Binary Random-Access Lists

A *random-access list*, also called a one-sided flexible array, is a data structure that supports array-like *lookup* and *update* functions, as well as the usual *cons*, *head*, and *tail* functions on lists. A signature for random-access lists is shown in Figure 6.4.

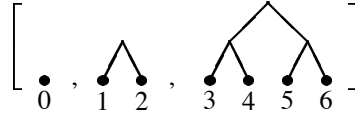
Kaldewaij and Dielissen [KD96] describe an implementation of random-access lists in terms of leftist left-perfect leaf trees. We can easily translate their implementation into the framework of numerical representations as a binary representation using complete binary leaf trees. A binary random-access list of size n thus contains a complete binary leaf tree for each 1 in the binary representation of n . The rank of each tree corresponds to the rank of the corresponding digit; if the i th bit of n is 1, then the random-access list contains a tree of size 2^i . For this example, we choose a dense representation, so the type of binary random-access lists is

```

datatype  $\alpha$  Tree = Leaf of  $\alpha$  | Node of int  $\times \alpha$  Tree  $\times \alpha$  Tree
datatype  $\alpha$  Digit = Zero | One of  $\alpha$  Tree
type  $\alpha$  RList =  $\alpha$  Digit list

```

The integer in each node is the size of the tree. This number is redundant since the size of every tree is completely determined by the size of its parent or by its position in the list of digits, but we include it anyway for convenience. Trees are stored in increasing order of size, and the order of elements (both within and between trees) is left-to-right. Thus, the head of the random-access list is the leftmost leaf of the smallest tree. Figure 6.5 shows a binary random-access list of size 7. Note that the maximum number of trees in a list of size n is $\lfloor \log(n+1) \rfloor$

Figure 6.5: A binary random-access list containing the elements $0 \dots 6$.

and the maximum depth of any tree is $\lfloor \log n \rfloor$.

Now, insertion into a binary random-access list (i.e., *cons*) is analogous to incrementing a binary number. Recall the increment function on dense binary numbers:

```
fun inc [] = [One]
    | inc (Zero :: ds) = One :: ds
    | inc (One :: ds) = Zero :: inc ds
```

To insert an element with *cons*, we first convert the element into a leaf, and then insert the leaf into the list of trees using a helper function *insTree* that follows the rules of *inc*.

```
fun cons (x, ts) = insTree (Leaf x, ts)
fun insTree (t, []) = [One t]
    | insTree (t, Zero :: ts) = One t :: ts
    | insTree (t1, One t2 :: ts) = Zero :: insTree (link (t1, t2), ts)
```

The *link* helper function is a pseudo-constructor for *Node* that automatically calculates the size of the new tree from the sizes of its children.

Deleting an element from a binary random-access list (using *tail*) is analogous to decrementing a binary number. Recall the decrement function on dense binary numbers:

```
fun dec [One] = []
    | dec (One :: ds) = Zero :: ds
    | dec (Zero :: ds) = One :: dec ds
```

Essentially, this function resets the first 1 to 0, while setting all the preceding 0s to 1s. The analogous operation on lists of trees is *borrowTree*. When applied to a list whose first digit has rank *r*, *borrowTree* returns a pair containing a tree of rank *r*, and the new list without that tree.

```
fun borrowTree [One t] = (t, [])
    | borrowTree (One t :: ts) = (t, Zero :: ts)
    | borrowTree (Zero :: ts) = let val (Node (_, t1, t2), ts') = borrowTree ts
    in (t1, One t2 :: ts') end
```

The *head* and *tail* functions “borrow” the leftmost leaf using *borrowTree* and then either return that leaf’s element or discard the leaf, respectively.

```
fun head ts = let val (Leaf x,  $\_$ ) = borrowTree ts in x end
fun tail ts = let val ( $\_$ , ts') = borrowTree ts in ts' end
```

The *lookup* and *update* functions do not have analogous arithmetic operations. Rather, they take advantage of the organization of binary random-access lists as logarithmic-length lists of logarithmic-depth trees. Looking up an element is a two-stage process. We first search the list for the correct tree, and then search the tree for the correct element. The helper function *lookupTree* uses the size field in each node to determine whether the *i*th element is in the left subtree or the right subtree.

```
fun lookup (Zero :: ts, i) = lookup (ts, i)
  | lookup (One t :: ts, i) =
    if i < size t then lookupTree (t, i) else lookup (ts, i - size t)
fun lookupTree (Leaf x, 0) = x
  | lookupTree (Node (w, t1, t2), i) =
    if i < w div 2 then lookupTree (t1, i) else lookupTree (t2, i - w div 2)
```

update works in same way but also reconstructs the path from the root to the updated leaf. This reconstruction is called *path copying* [ST86a] and is necessary for persistence.

```
fun update (Zero :: ts, i, y) = Zero :: update (ts, i, y)
  | update (One t :: ts, i, y) =
    if i < size t then One (updateTree (t, i, y)) :: ts else One t :: update (ts, i - size t, y)
fun updateTree (Leaf x, 0, y) = Leaf y
  | updateTree (Node (w, t1, t2), i, y) =
    if i < w div 2 then Node (w, updateTree (t1, i, y), t2)
    else Node (w, t1, updateTree (t2, i - w div 2, y))
```

The complete code for this implementation is shown in Figure 6.6.

cons, *head*, and *tail* perform at most $O(1)$ work per digit and so run in $O(\log n)$ worst-case time. *lookup* and *update* take at most $O(\log n)$ time to find the right tree, and then at most $O(\log n)$ time to find the right element in that tree, for a total of $O(\log n)$ worst-case time.

6.2.2 Binomial Heaps

Binomial queues [Vui78, Bro78] are a classical implementation of mergeable priority queues. To avoid confusion with FIFO queues, we will henceforth refer to priority queues as *heaps* and binomial queues as *binomial heaps*. Heaps support four main functions: inserting an element

```

structure BinaryRandomAccessList : RANDOMACCESSLIST =
struct
  datatype  $\alpha$  Tree = Leaf of  $\alpha$  | Node of  $\text{int} \times \alpha \text{ Tree} \times \alpha \text{ Tree}$   (* int is size of tree *)
  datatype  $\alpha$  Digit = Zero | One of  $\alpha \text{ Tree}$ 
  type  $\alpha$  RList =  $\alpha$  Digit list
  exception EMPTY and INDEX
  val empty = []
  fun isEmpty  $ts$  = null  $ts$ 
  fun size (Leaf  $x$ ) = 1
    | size (Node ( $w, t_1, t_2$ )) =  $w$ 
  fun link ( $t_1, t_2$ ) = Node (size  $t_1$  + size  $t_2, t_1, t_2$ )
  fun insTree ( $t, []$ ) = [One  $t$ ]
    | insTree ( $t, \text{Zero} :: ts$ ) = One  $t :: ts$ 
    | insTree ( $t_1, \text{One } t_2 :: ts$ ) = Zero :: insTree (link ( $t_1, t_2$ ),  $ts$ )
  fun borrowTree [] = raise EMPTY
    | borrowTree [One  $t$ ] = ( $t, []$ )
    | borrowTree (One  $t :: ts$ ) = ( $t, \text{Zero} :: ts$ )
    | borrowTree (Zero ::  $ts$ ) = let val (Node ( $\_, t_1, t_2$ ),  $ts'$ ) = borrowTree  $ts$ 
      in ( $t_1, \text{One } t_2 :: ts'$ ) end
  fun cons ( $x, ts$ ) = insTree (Leaf  $x, ts$ )
  fun head  $ts$  = let val (Leaf  $x, \_$ ) = borrowTree  $ts$  in  $x$  end
  fun tail  $ts$  = let val ( $\_, ts'$ ) = borrowTree  $ts$  in  $ts'$  end
  fun lookupTree (Leaf  $x, 0$ ) =  $x$ 
    | lookupTree (Leaf  $x, i$ ) = raise INDEX
    | lookupTree (Node ( $w, t_1, t_2$ ),  $i$ ) =
      if  $i < w \text{ div } 2$  then lookupTree ( $t_1, i$ ) else lookupTree ( $t_2, i - w \text{ div } 2$ )
  fun updateTree (Leaf  $x, 0, y$ ) = Leaf  $y$ 
    | updateTree (Leaf  $x, i, y$ ) = raise INDEX
    | updateTree (Node ( $w, t_1, t_2$ ),  $i, y$ ) =
      if  $i < w \text{ div } 2$  then Node ( $w, \text{updateTree } (t_1, i, y), t_2$ )
      else Node ( $w, t_1, \text{updateTree } (t_2, i - w \text{ div } 2, y)$ )
  fun lookup ([],  $i$ ) = raise INDEX
    | lookup (Zero ::  $ts, i$ ) = lookup ( $ts, i$ )
    | lookup (One  $t :: ts, i$ ) =
      if  $i < \text{size } t$  then lookupTree ( $t, i$ ) else lookup ( $ts, i - \text{size } t$ )
  fun update ([],  $i, y$ ) = raise INDEX
    | update (Zero ::  $ts, i, y$ ) = Zero :: update ( $ts, i, y$ )
    | update (One  $t :: ts, i, y$ ) =
      if  $i < \text{size } t$  then One (updateTree ( $t, i, y$ )) ::  $ts$  else One  $t :: \text{update } (ts, i - \text{size } t, y)$ 
end

```

Figure 6.6: Binary random-access lists.

```

signature ORDERED =
sig
  type T                (* type of ordered elements *)
  val leq : T × T → bool (* total ordering relation *)
end

signature HEAP =
sig
  structure Elem : ORDERED

  type Heap

  exception EMPTY

  val empty      : Heap
  val isEmpty    : Heap → bool

  val insert      : Elem.T × Heap → Heap
  val merge       : Heap × Heap → Heap

  val findMin     : Heap → Elem.T (* raises EMPTY if heap is empty *)
  val deleteMin  : Heap → Heap   (* raises EMPTY if heap is empty *)
end

```

Figure 6.7: Signature for heaps.

(*insert*), merging two heaps (*merge*), finding the minimum element (*findMin*), and deleting the minimum element (*deleteMin*). A Standard ML signature for heaps appears Figure 6.7.

Remark: Heaps are similar to the sortable collections of Section 3.5.2, but use a different mechanism for specifying the desired comparison function. For sortable collections, the comparison function is supplied when a new object is created, and every object can have a different comparison function. This approach is very flexible, but causes problems in the presence of an function that combines two objects, such as *merge*. If the two objects being merged have different comparison functions, which should the resulting object keep? To avoid this ambiguity, we fix the comparison function (and therefore the type of elements being compared) when the Standard ML structure implementing heaps is created. Then, we can be sure that any two objects being merged shared the same comparison function. ◇

In the framework of numerical representations, binomial heaps are a binary representation with heap-ordered, binomial trees. A tree is *heap-ordered* if the element at every node is smaller than the elements at any of its children, with ties broken arbitrarily. As with binary random-access lists, binomial heaps contain one tree for each 1 in the binary representation of the size of the heap, and the trees have the same weights as their matching digits.

Assuming an ORDERED structure *Elem* that specifies the element type and comparison function, the types of binomial trees and binomial heaps can be written as follows:

```
datatype Tree = Node of int  $\times$  Elem.T  $\times$  Tree list
type Heap = Tree list
```

This time, we have chosen a sparse representation, where the integer at each node is the rank of the tree. For reasons that will become clear later, we maintain the list of trees representing a heap in increasing order of rank, but maintain the list of trees representing the children of a node in decreasing order of rank.

Remark: The rank information on each node that is not a root is redundant since the i th child of a node of rank r always has rank $r - i$. However, we maintain this information anyway because doing so simplifies the code slightly. $\diamond$

The fundamental operation on binomial trees is *link*, which compares the roots of two trees of rank r and makes the tree with the larger root a child of the tree with the smaller root, producing a tree of rank $r + 1$.

```
fun link ( $t_1$  as Node ( $r, x_1, c_1$ ),  $t_2$  as Node ( $-, x_2, c_2$ )) =
  if Elem.leq ( $x_1, x_2$ ) then Node ( $r+1, x_1, t_2 :: c_1$ ) else Node ( $r+1, x_2, t_1 :: c_2$ )
```

Since the children of a tree are maintained in decreasing order of rank, adding the new child to the list takes only $O(1)$ time.

Now, inserting an element into a binomial heap is similar to the increment function on sparse binary numbers. Whenever we find two trees of the same rank, we *link* them and reinsert the linked tree into the list. This corresponds to a carry in binary arithmetic. We use the *insTree* helper function to insert new trees into the list of trees; *insert* builds a new singleton tree and calls *insTree*.

```
fun insTree ( $t, []$ ) = [ $t$ ]
  | insTree ( $t_1, ts$  as  $t_2 :: rest$ ) =
    if rank  $t_1$  < rank  $t_2$  then  $t_1 :: ts$  else insTree (link ( $t_1, t_2$ ),  $rest$ )
fun insert ( $x, ts$ ) = insTree (Node (0,  $x, []$ ),  $ts$ )
```

merge is similar to addition on sparse binary numbers, where again we *link* trees of equal rank whenever there is a carry.

```
fun merge ( $ts_1, []$ ) =  $ts_1$ 
  | merge ( $[], ts_2$ ) =  $ts_2$ 
  | merge ( $t_1 :: ts_1, t_2 :: ts_2$ ) =
    if rank  $t_1$  < rank  $t_2$  then  $t_1 :: merge (ts_1,  $t_2 :: ts_2$ )$ 
    else if rank  $t_2$  < rank  $t_1$  then  $t_2 :: merge (t_1 :: ts_1,  $ts_2$ )$ 
    else insTree (link ( $t_1, t_2$ ), merge ( $ts_1, ts_2$ ))
```

Since every tree is heap-ordered, we know that the minimum element within any given tree is the root. However, we do not know which tree has the minimum root, so *findMin* scans all the roots in the heap.

```
fun findMin [t] = root t
  | findMin (t :: ts) = let val x = root t
                      val y = findMin ts
                      in if Elem.leq (x, y) then x else y end
```

Finally, *deleteMin* begins by removing the tree with the minimum root. (In the case of ties, we should take care to remove the tree with the same root as returned by *findMin*.) Once we have discarded the root of this tree, we are left with two lists of trees: one representing the children of the discarded tree, and one representing the remaining trees in the heap. To obtain a single heap, we simply merge these two lists, but since the lists are maintained in opposite orders, we first reverse the list of children.

```
fun deleteMin ts =
  let fun getMin [t] = (t, [])
      | getMin (t :: ts) =
        let val (t', ts') = getMin ts
        in if Elem.leq (root t, root t') then (t, ts) else (t', t :: ts') end
        val (Node (_, x, ts1), ts2) = getMin ts
        in merge (rev ts1, ts2) end
```

The complete implementation of binomial heaps appears in Figure 6.8. Since heaps contain no more than $\lfloor \log(n+1) \rfloor$ trees, and binomial trees have no more than $\lfloor \log n \rfloor$ children, each of these functions takes $O(\log n)$ worst-case time.

6.3 Segmented Binary Numbers

We next explore two variations of binary numbers that allow a number to be incremented or decremented in $O(1)$ worst-case time. Basing a numerical representation on these variations, rather than ordinary binary numbers, reduces the running time of many insertion and deletion functions from $O(\log n)$ to $O(1)$. First, we present a somewhat complicated representation and sketch the design of random-access lists and heaps based on this representation. In the next section, we present a much simpler representation that is usually superior in practice.

The problem with ordinary binary numbers is that carries and borrows can cascade. For example, incrementing $2^k - 1$ causes k carries in binary arithmetic. Symmetrically, decrementing 2^k causes k borrows. *Segmented binary numbers* solve this problem by allowing multiple carries or borrows to be executed in a single step.